

Student Tutorial 1: Git and GitHub Basics

What you will learn

- open a terminal or Bash window
- check whether Git is installed
- create a small research project folder
- initialize a Git repository
- make and inspect commits
- understand what should and should not be tracked

Why a hybrid approach?

We use the terminal for Git commands, but a text editor to write files. This avoids differences between Mac/Linux (Bash) and Windows (PowerShell), and makes the exercise easier.

Step 1: Open a terminal / Bash window

A terminal is a window where you type commands instead of clicking with the mouse.

- **Mac**
 1. Press **Cmd (⌘) + Space**
 2. Type **Terminal**
 3. Press **Enter**

A window will open with text — this is your terminal.
- **Linux**
 1. Open the application menu
 2. Search for **Terminal**
 3. Open it
- **Windows**

Option A (recommended): Git Bash

If you installed Git:

1. Click **Start**
2. Type **Git Bash**
3. Open it

This may not work due to administrator permissions

- **Option B: Windows Terminal or PowerShell**
 1. Click **Start**
 2. Type **Terminal** or **PowerShell**
 3. Open it

Before you begin: do you have Git installed?

Git is the tool we use to track changes in a project over time. It is best if Git is installed before the session, but do not worry if installation does not work during class. You can pair up and try the steps yourself later.

Open a terminal and type:

```
git --version
```

If Git is installed, you should see something like:

```
git version 2.54.0
```

If Git is not installed

- Download at <https://git-scm.com/install/>

Step 2: Move to a good working location if you aren't at one

```
pwd
```

```
(cd directory_name if you need to change to another directory)
```

Useful command meanings: cd = change directory (move into another folder); pwd = print working directory

Step 3: Create a new project folder

```
mkdir analysis_project
```

```
cd analysis_project
```

```
git init
```

Useful command meanings: mkdir = create new folder, git init = start version control

Step 4: Create the basic project structure (folders and files)

In this step, you will:

- Use the terminal to create folders
- Use a text editor to create and edit files

4.1 Create folders

Make sure you are in the correct folder (in the terminal)

```
pwd
```

If needed:

```
cd analysis_project
```

Make new directories

```
mkdir data
```

```
mkdir results
```

4.2 Create files

We now switch from terminal to editor to create and edit files

Open the folder in a text editor (recommended: VS Code)

Option A (open via terminal):

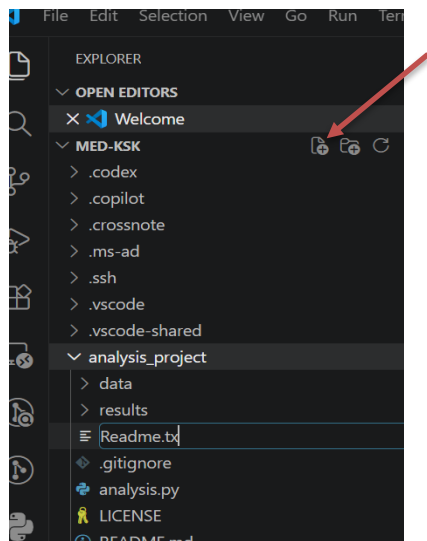
```
code .
```

Option B (open manually):

Open your editor → go to Folder 'analysis_project'

In VS code you can see the file structure on the left upper side and add files from there

Create files (in the editor, right click → New File or click on the New file icon)



Type in this (including the extension, e.g. .md) to add new files one at a time

- README.md
- analysis.py
- requirements.txt
- .gitignore
- LICENSE

Inside the data folder, create:

- README.txt

4.3 Check your structure

```
analysis_project/  
README.md  
analysis.py  
requirements.txt  
.gitignore  
LICENSE  
data/  
  README.txt  
results/
```

Why we do this:

We use the terminal for structure and commands, and the editor for writing. This reflects real research workflows.

You can also create these files using the file explorer if you prefer.

If you want to do this using commands:

Mac / Linux / Git Bash

```
touch README.md  
  
touch analysis.py  
  
touch requirements.txt  
  
touch .gitignore  
  
touch LICENSE  
  
mkdir data  
  
touch data/README.txt  
  
mkdir results
```

Windows (Terminal/PowerShell)

```
New-Item README.md
```

```
New-Item analysis.py
New-Item requirements.txt
New-Item .gitignore
New-Item LICENSE
New-Item -ItemType Directory data
New-Item data\README.txt
New-Item -ItemType Directory results
```

Your project now contains a minimal shareable structure:

```
analysis_project/
├─ README.md
├─ LICENSE
├─ analysis.py
├─ requirements.txt
├─ .gitignore
├─ data/
│   └─ README.txt
└─ results/
```

Step 5: Add content to the files (hybrid method)

We now add simple content so Git has something to track.

- README.md explains what the project is and how to use it.
- analysis.py contains the example code.
- requirements.txt lists software dependencies.
- .gitignore lists files or folders that should not be tracked.
- LICENSE states reuse conditions.
- data/README.txt explains the data folder, even if data cannot be shared.
- results/ is where outputs may be written; generated outputs are often not committed.

Open each file in your text editor, paste the content below. Make sure you **SAVE** each file (Ctrl+S on Windows, Cmd+S on Mac) before continuing.

README.md

```
# Analysis Project
```

This is a simple example project for learning Git and reusable research code.

Files

- analysis.py: example analysis script
- data/README.txt: information about the data

How to run

```
python analysis.py
```

analysis.py

```
print("Hello from the analysis script")
```

data/README.txt

Data are not included in this example project.

This file explains what the data folder is for.

requirements.txt

Add required packages here, one per line.

Example: pandas

.gitignore

results/

*.log

.env

LICENSE

License not yet selected. Reuse conditions not specified.

Important:

Git tracks changes in files. If files are empty or not saved, Git has nothing meaningful to track.

Step 6: Check before adding files to Git

Check that you are not about to track files that should stay private or local.

- Do not commit personal or sensitive data.

- Do not commit passwords, API keys, tokens, or credentials.
- Do not commit data you are not allowed to redistribute.
- Do not commit large generated files unless there is a clear reason.
- Use .gitignore for files and folders that should stay local.

Step 7: Check Git status

```
git status
```

Git will show files that are untracked or changed.

Step 8: Add files to Git

```
git add .  
git status
```

Step 9: Make your first commit

```
git commit -m "Initial commit"
```

If Git asks for your name or email:

```
git config --global user.name "Your Name"  
git config --global user.email "your.email@example.com"  
git commit -m "Initial commit"
```

A commit is a saved checkpoint in the history of your project.

Step 10: Make and commit a small change

Open README.md and add one more line at the end:

```
This project was created in an Open Science course.
```

Then run:

```
git status  
git add README.md  
git commit -m "Add course note to README"
```

Step 11: View the history

```
git --no-pager log --oneline
```

What you should take away

- Git helps track changes over time.
- A commit is a named checkpoint.

- A small project structure can already support reuse.